

**Московский государственный технический  
университет им. Н. Э. Баумана**

Факультет «Информатика и системы управления»  
Кафедра ИУ5 «Системы обработки информации и управления»

Курс «Парадигмы и конструкции языков программирования»  
Отчет по лабораторной работе №4  
«Модульное тестирование»

Выполнил:  
Студент группы ИУ5-31Б  
Чичёв Иван

Проверил:  
Гапанюк Ю. Е.

2025 г.

### Задание

1. Выберите любой фрагмент кода из лабораторных работ 1 или 2 или 3-4.
2. Модифицируйте код таким образом, чтобы он был пригоден для модульного тестирования.
3. Разработайте модульные тесты. В модульных тестах необходимо применить следующие технологии:
  - TDD - фреймворк (не менее 3 тестов).
  - BDD - фреймворк (не менее 3 тестов).

main.py

```
from lab_python_oop.Rectangle import Rectangle
from lab_python_oop.Circle import Circle
from lab_python_oop.Square import Square
from colorama import Fore, Style, init

def create_shapes(size):
    return [
        Rectangle(size, size, "синий"),
        Circle(size, "зеленый"),
        Square(size, "красный")
    ]

def format_shapes(shapes):
    init(autoreset=True)
    colors = [Fore.BLUE, Fore.GREEN, Fore.RED]
    return [c + repr(s) + Style.RESET_ALL for c, s in zip(colors, shapes)]

def main():
    shapes = create_shapes(21)
    for line in format_shapes(shapes):
        print(line)

if __name__ == "__main__":
    main()
```

testShapes.py

```
import unittest
from lab_python_oop.Rectangle import Rectangle

class TestRectangle(unittest.TestCase):
    def testArea(self):
        rectangle = Rectangle(3, 4, "синий")
        self.assertEqual(rectangle.area(), 12)

    def testColor(self):
        rect = Rectangle(3, 4, "красный")
        self.assertEqual(str(rect.color), "красный")

    def testRepr(self):
        rectangle = Rectangle(3, 4, "красный")
        reprString = repr(rectangle)
        self.assertIn("красный", reprString)
        self.assertIn("Прямоугольник", reprString)

if __name__ == "__main__":
    unittest.main()
```

shapes.feature

Feature: Форматирование и создание фигур

Scenario: Проверка создания и форматирования фигур

Given фигуры созданы с размером 10

When форматирую фигуры для вывода

Then результат содержит фигуры с ожидаемыми параметрами

```
"""
{
    "Прямоугольник": "синий",
    "Круг": "зеленый",
    "Квадрат": "красный"
}
"""
```

test\_shapes\_bdd.py

```
import json
from behave import given, when, then
from main import create_shapes, format_shapes
```

```

@given('фигуры созданы с размером 10')
def step_given_shapes_created(context):
    context.shapes = create_shapes(10)

@when('форматирую фигуры для вывода')
def step_when_format_shapes(context):
    context.formatted = format_shapes(context.shapes)

@then('результат содержит фигуры с ожидаемыми параметрами')
def step_then_check_lines(context):
    expected_lines = json.loads(context.text)
    formatted = ''.join(context.formatted)

    for shape_name, color in expected_lines.items():
        assert (
            shape_name in formatted,
            f'Нет фигуры {shape_name} в выводе'
        )

        assert (
            color in formatted,
            f'Нет цвета {color} для {shape_name}'
        )

```

```

command python -m pip install python3 -m python3
● van@DESKTOP-260AC2V:~/git/chichev-BKIT-labs-2025$ python3 -m unittest discover
...
-----
Ran 3 tests in 0.000s

OK

```